

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Centro Regional de Colón

Facultad de Ingeniería de Sistemas Computacionales

Laboratorio #3 – Algoritmo Quicksort

Asignatura:

Estructura de Datos II

Facilitador:

Ing. Ariel Martínez

Estudiante: Chevesade Ornano

Fecha:

18 de mayo de 2026

Grupo: 5GS241

1. Funcionamiento del Algoritmo Quicksort

Quicksort es un algoritmo de ordenamiento eficiente basado en la estrategia de diseño divide y vencerás, propuesto por Tony Hoare en 1959. Su principio fundamental consiste en seleccionar un elemento llamado pivote y reorganizar los demás elementos del arreglo de manera que todos los menores al pivote queden a su izquierda y todos los mayores a su derecha. Este proceso se repite recursivamente en cada sub-arreglo hasta que el arreglo completo queda ordenado.

1.1 Fases del algoritmo

- **DIVIDIR:** Se selecciona un pivote (en esta implementación, el último elemento del sub-arreglo) y se llama a la función `particion()`, la cual reorganiza los elementos y ubica el pivote en su posición definitiva.
- **CONQUISTAR:** Se aplica Quicksort de forma recursiva al sub-arreglo izquierdo (elementos menores al pivote) y al sub-arreglo derecho (elementos mayores al pivote).
- **COMBINAR:** No se requiere ninguna acción adicional, ya que el ordenamiento se realiza directamente sobre la misma lista (in-place).

1.2 Esquema de partición (Lomuto)

La implementación utiliza el esquema de partición de Lomuto, que funciona de la siguiente manera:

- Se toma el último elemento del sub-arreglo como pivote.
- Se recorre el sub-arreglo con un índice j desde el inicio hasta antes del pivote.
- Un segundo índice i lleva la cuenta de los elementos menores o iguales al pivote.
- Cada vez que se encuentra un elemento $\leq$ pivote, se incrementa i y se intercambia `lista[i]` con `lista[j]`.
- Al finalizar el recorrido, el pivote se intercambia con `lista[i+1]`, quedando así en su posición final correcta.

1.3 Análisis de complejidad

Caso	Complejidad Temporal	Descripción
Mejor caso	$O(n \log n)$	Pivote siempre central
Caso promedio	$O(n \log n)$	Distribución aleatoria
Peor caso	$O(n^2)$	Lista ya ordenada
Espacio (pila)	$O(\log n)$	Recursión promedio

El peor caso ocurre cuando el pivote seleccionado siempre es el elemento máximo o mínimo del sub-arreglo (por ejemplo, en una lista ya ordenada). En la práctica, con datos aleatorios, el caso promedio de $O(n \log n)$ es el esperado, lo cual lo hace uno de los algoritmos de ordenamiento más rápidos en uso general.

2. Lenguaje Utilizado: Python 3

El programa fue desarrollado en Python 3, lenguaje de alto nivel, interpretado y de propósito general. Python fue seleccionado por las siguientes razones:

- Sintaxis clara: su estructura legible permite que los comentarios explicativos del algoritmo sean fácilmente comprensibles.
- Tipado dinámico: facilita la manipulación de listas sin necesidad de declarar tipos explícitamente.
- Intercambio en una línea: la expresión `a, b = b, a` permite realizar swaps de forma elegante y sin variable temporal.
- Módulos estándar: se utilizaron `random` (generación de números aleatorios) y `time` (medición del tiempo de ejecución), ambos incluidos en la biblioteca estándar.

La implementación no hace uso de las funciones incorporadas `sort()` ni `sorted()` del lenguaje, cumpliendo con el requisito funcional del laboratorio. El algoritmo Quicksort fue codificado íntegramente de forma manual.

3. Resultados Obtenidos

Al ejecutar el programa se generó automáticamente una lista de 20 números enteros aleatorios en el rango de 1 a 100. La salida del programa presentó claramente:

- La lista original con los números en su orden aleatorio inicial.
- La lista ordenada de menor a mayor, resultado de la aplicación del algoritmo Quicksort.
- Estadísticas adicionales: valor mínimo, valor máximo y tiempo de ejecución en milisegundos.

El tiempo de ejecución registrado fue extremadamente bajo (del orden de 0.03 ms para $n = 20$), lo que demuestra la alta eficiencia del algoritmo para conjuntos de datos pequeños. Este resultado es consistente con la complejidad teórica $O(n \log n)$ en el caso promedio.

La verificación del resultado se realizó de forma visual, confirmando que cada elemento de la lista ordenada es mayor o igual al anterior, lo que garantiza el correcto funcionamiento del algoritmo implementado.

4. Dificultades Encontradas

Durante el desarrollo del laboratorio se identificaron las siguientes dificultades:

- **Entender el rol del índice i como marcador de la frontera entre elementos menores y mayores al pivote requirió trazar manualmente varios ejemplos paso a paso antes de que la lógica quedara completamente clara.** Comprensión del índice i en la partición:
- **Determinar el momento exacto en que el pivote debe colocarse en su posición final (el swap al terminar el recorrido) fue un punto de confusión inicial, ya que hacerlo en el momento incorrecto altera el resultado.** Posicionamiento correcto del pivote:
- **Identificar correctamente el caso base (bajo $\geq$ alto) fue fundamental para evitar recursión infinita o acceso a índices fuera de rango.** Caso base de la recursión:
- **Dado que Quicksort ordena in-place, fue necesario hacer una copia de la lista antes de ordenarla para poder mostrar tanto la versión original como la ordenada.** Preservar la lista original:

5. Conclusión

La implementación del algoritmo Quicksort permitió comprender de manera práctica el paradigma de divide y vencerás aplicado al ordenamiento de datos. A través del desarrollo del programa en Python, se evidenció que Quicksort es un algoritmo eficiente tanto en tiempo como en uso de memoria, siendo preferible a otros algoritmos como Bubble Sort o Insertion Sort para conjuntos de datos de tamaño moderado a grande.

El ejercicio reforzó la importancia de comprender la complejidad algorítmica no solo de forma teórica, sino también a través de la observación empírica del tiempo de ejecución. El laboratorio cumplió satisfactoriamente con todos los requisitos funcionales y técnicos establecidos en la guía.